Electricity Load Forecast

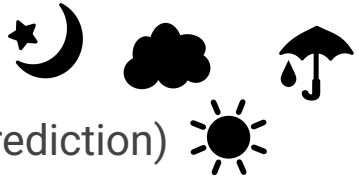
real data from the dairy processing industry

Marius Gören
Ironhack DSMLJAN2026
March 13th 2026

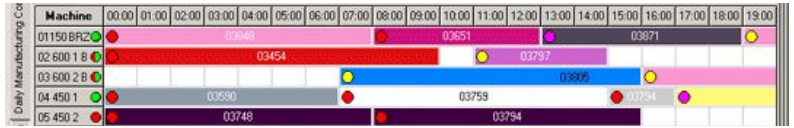
Overview of the goal

<u>Forecast of the electricity load for the next 15 min (24 h)</u>

1. using only the historical data of the electricity load
   - adding many features regarding date/time (shift, holiday etc.)
2. include weather data



   - historical (used for training)
   - forecast (used for future load prediction)
3. including step numbers of the production machines
   - forecast based on the production plan for the future



15 minute forecast - why?

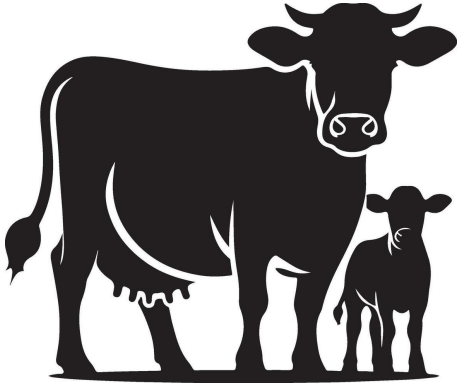
“**Load management** means that companies **avoid** very high **electricity peaks** by shifting or **reducing** their **power** use at certain times. If they keep their **peak low**, they can **save hundreds of thousands of euros** because the grid operator gives back a large part of the grid fees.”

The data sets

electricity load

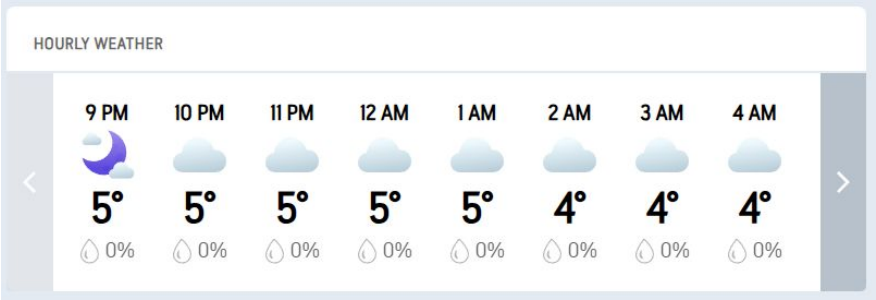
data set from a real industrial company

- dairy production (milk, butter, milk powder)
- nothing but the electricity load since 01.01.2024
  - one value each 15 minutes (quarterly average values)
  - more than 75,000 rows



weather data

- imported using a free API to openmeteo weather
  - latitude/longitude
  - start/end date (matching the period of electricity load data set)
- hourly values for
  - temperature
  - relative humidity
  - wind speed
  - radiation
  - precipitation
  - clouds
- resampled/interpolate to quarterly values

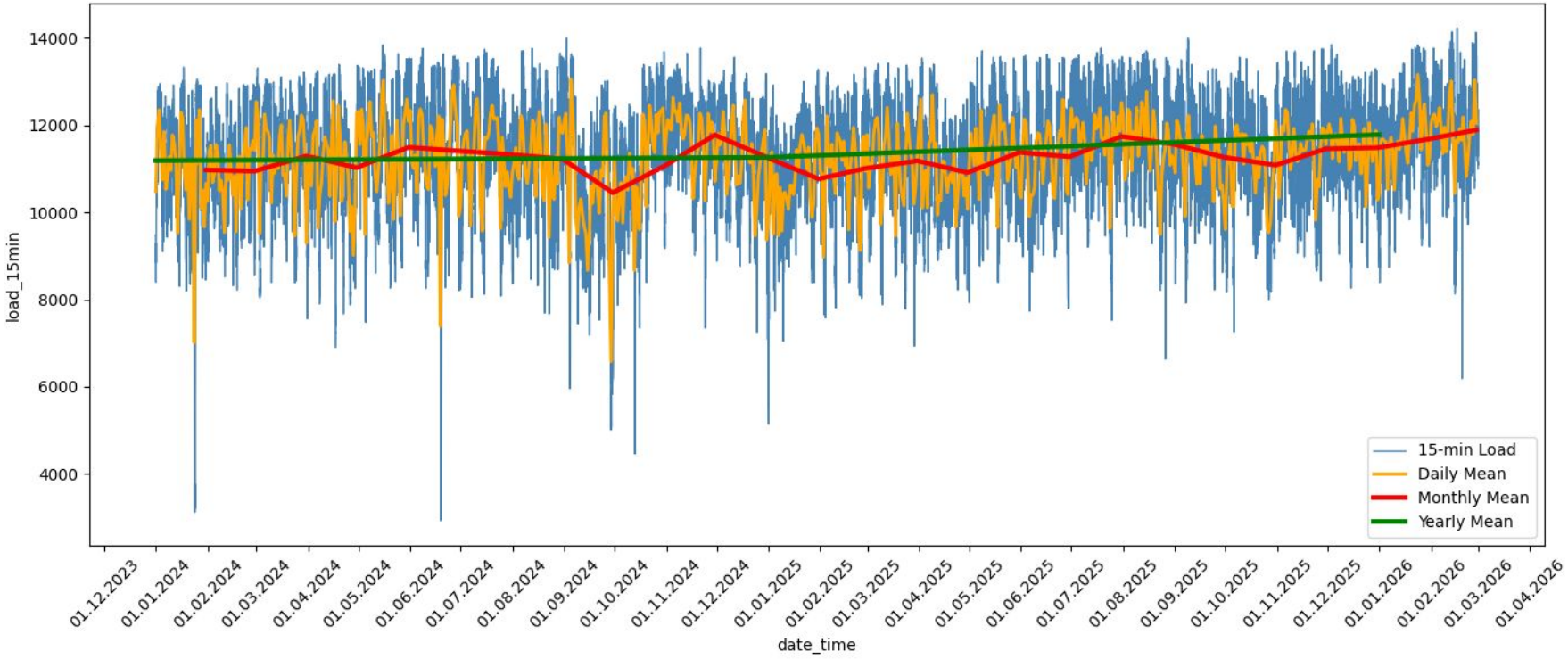


Exploratory Data Analysis

EDA

- convert electricity load from string to int
- change unit from MW to kW
- fill NaN-gaps linearly
- remove outliers (values > 15 MW -> not possible for the plant)
- use UTC instead of CET
  - otherwise you need to
    - remove duplicated timestamps
    - fill missing timestamps
    - caused by time shift (summertime <-> wintertime)

Exploring the electricity load

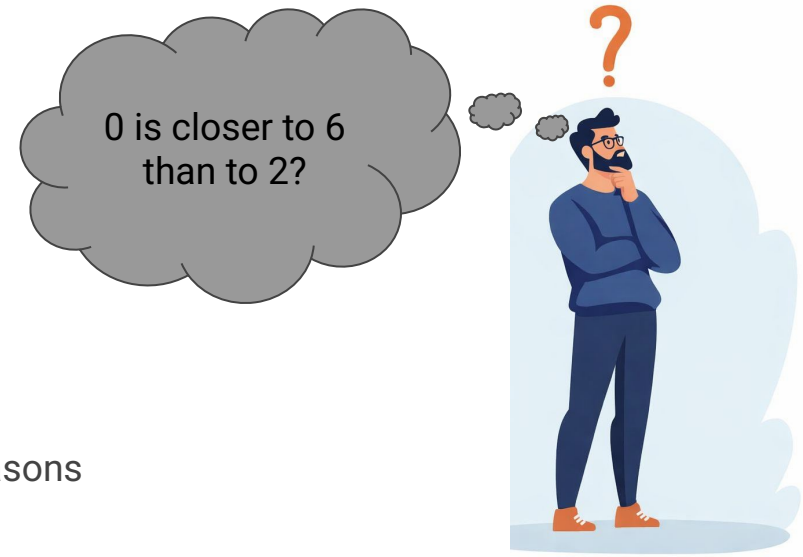


Feature engineering

electricity load data set

- Convert date/time to:
  - hour
  - minute
  - time (in minutes)
  - month
  - season
- but the model needs to know that
  - monday (day 0) is closer to sunday (day 6)
  - also for the hours, minutes, months and seasons

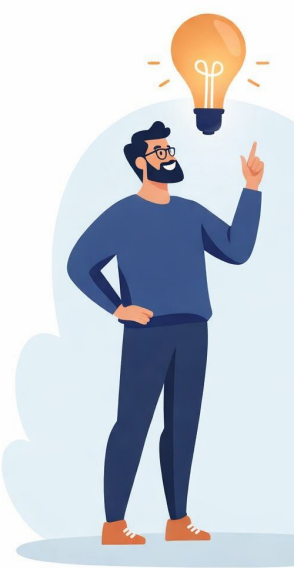
-> they are **useless…**

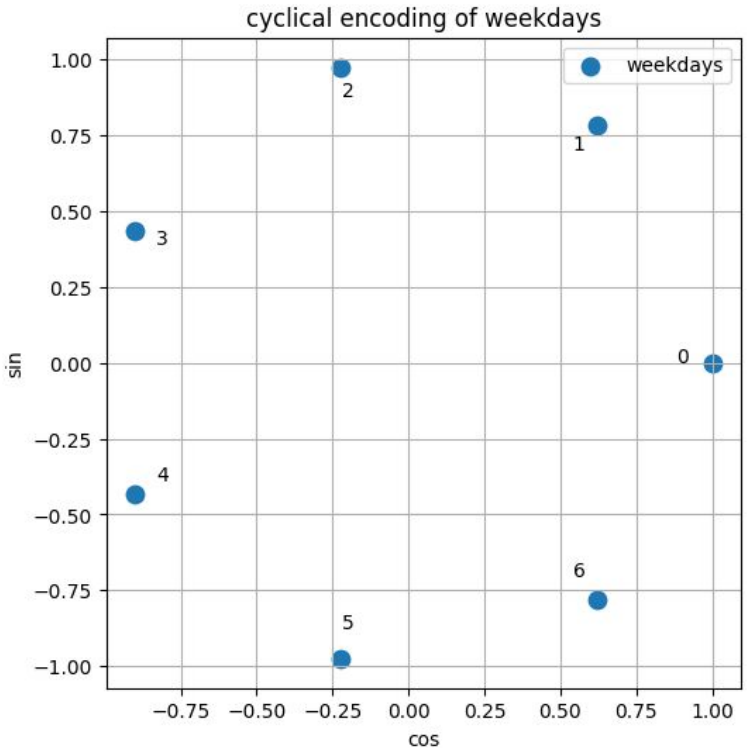


electricity load data set

… until you use **cyclical encoding!**

- all these features, they are cycles
- after reaching the highest number, they start again at 0
- calculate the sine and cosine
  - visualize to see that it is true





electricity load data set

Adding more columns:

- weekend (saturday/sunday)
- holidays (based on national holidays in germany)
  - python library holidays -> select germany and the province
  - based on that -> create bridging days
- shifts (early/late/night)
- shift change (± 1 hour -> e.g. 1 p.m. to 2 p.m.)
- rolling mean (3h, 6h, 12h)
- rolling std (3h, 6h, 12h)
- load x hours ago (15min, 30min, 1h, 2h, 24h, 168h)

weather data

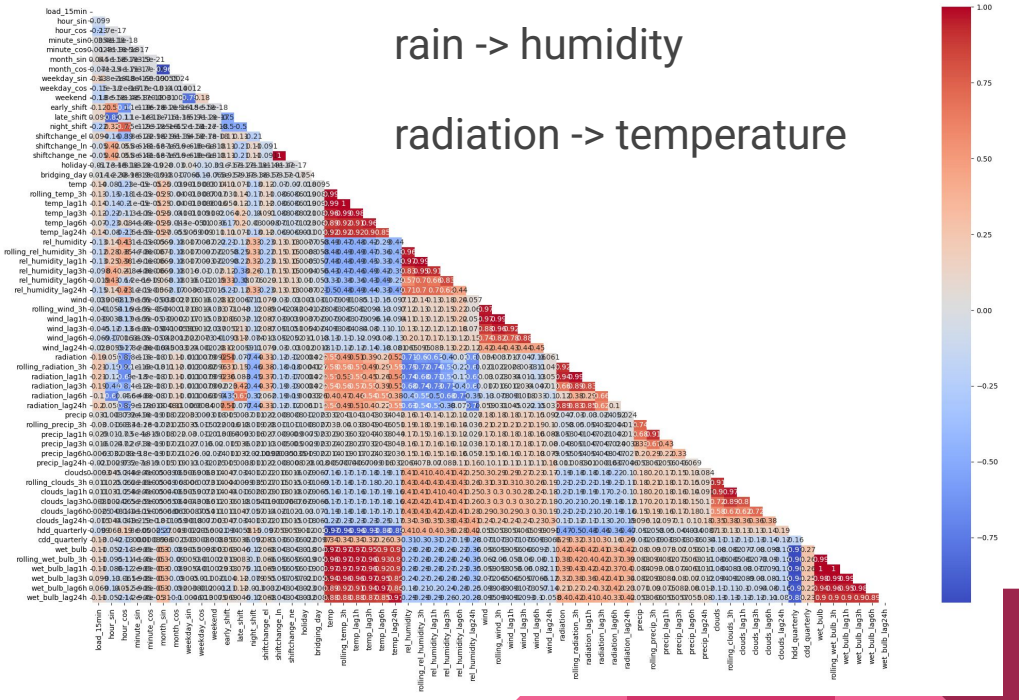
calculating new features based on the weather:

- for all the weather data -> create rolling weather and lags
  - lag (1h, 3h, 6h, 24h, 48h) and rolling weather(3h, 6h, 12h)
- HDD -> heating degree days
  - daily, quarterly
- CDD -> cooling degrees days
  - daily, quarterly
- wet bulb temperature (cooling limit temperature)
  - lag (1h, 3h, 6h, 24h, 48h) and rolling (3h, 6h, 12h)

The decision (which features are necessary)

checking multi-collinearity:

- drop some features
  - hour, minute, month, weekday
    - keep only sin/cos
  - season completely
    - we already have month
  - weather lags
    - keep only <=24h
  - rolling mean of weather
    - keep only 3h



rain -> humidity

radiation -> temperature

the remaining collinearity is only physical in nature !

XGBoost

Hyperparameter tuning

using Optuna:

- much faster than RandomizedSearchCV
- using ranges and not preselected values for each hyperparameter
- learns which combinations are better together and focus on them

I decide to create an automatic pipeline with that:

1. rough tuning -> using a big range for each hyperparameter
2. fine tuning -> using the best parameters from rough tuning
    a. creating a range of ±20% around the best parameters

XGBoost | including weather data

using **data** including the **timestamp, historical electricity load** and **weather**

- predict the next 15 min load with an **accuracy** of:
  - rel. MAE = **2.10 %** <- **2.12 %** (without weather data)
  - ***rel. RMSE*** = **2.72 %** <- **2.73 %** (without weather data)
  - RMSE = 306,6 kW <- 309,2 kW
  - R^2 score = 91,33 % <- 91,23 %

- **Only a slight improvement using the weather data!**
- **I prefer to choose relative RMSE, because I want to predict load peaks as accurately as possible!**

	R^2	RMSE	MAE	relRMSE	relMAE
optimal	>70%	0	0	0%	0%
unit	%	kW	kW	%	%
train	96.21	244.07	188.64	2.18	1.69
test	91.33	306.64	237.17	2.72	2.1
diff	4.88	-62.57	-48.53	-0.54	-0.42

XGBoost | without rolling load/load lags

using **data** including the **timestamp** and **weather**

- predict the next 15 min load with an **accuracy** of:
  - rel. MAE = **6.71 %** <- **2.12 %** (without weather data)
  - ***rel. RMSE*** = **8.41 %** <- **2.73 %** (without weather data)
  - RMSE = 947,9 kW <- 309,2 kW
  - R^2 score = 17,13 % <- 91,23 %

- **Really bad result without rolling load and load lags!**

	R^2	RMSE	MAE	relRMSE	relMAE
optimal	>70%	0	0	0%	0%
unit	%	kW	kW	%	%
train	16.43	1146.21	885.6	10.25	7.92
test	17.13	947.98	756.91	8.41	6.71
diff	-0.7	198.23	128.7	1.84	1.21

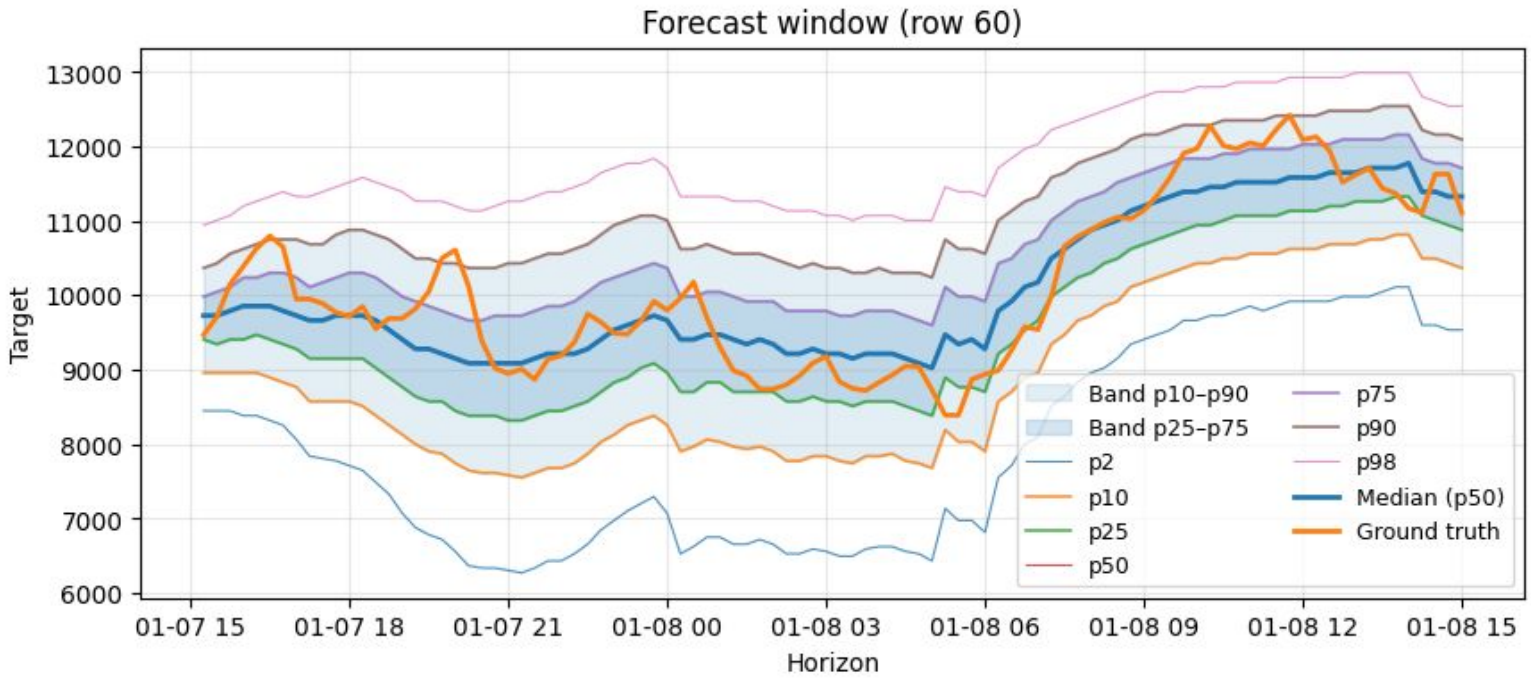
Temporal Fusion Transformer

Deep learning model designed for forecasting

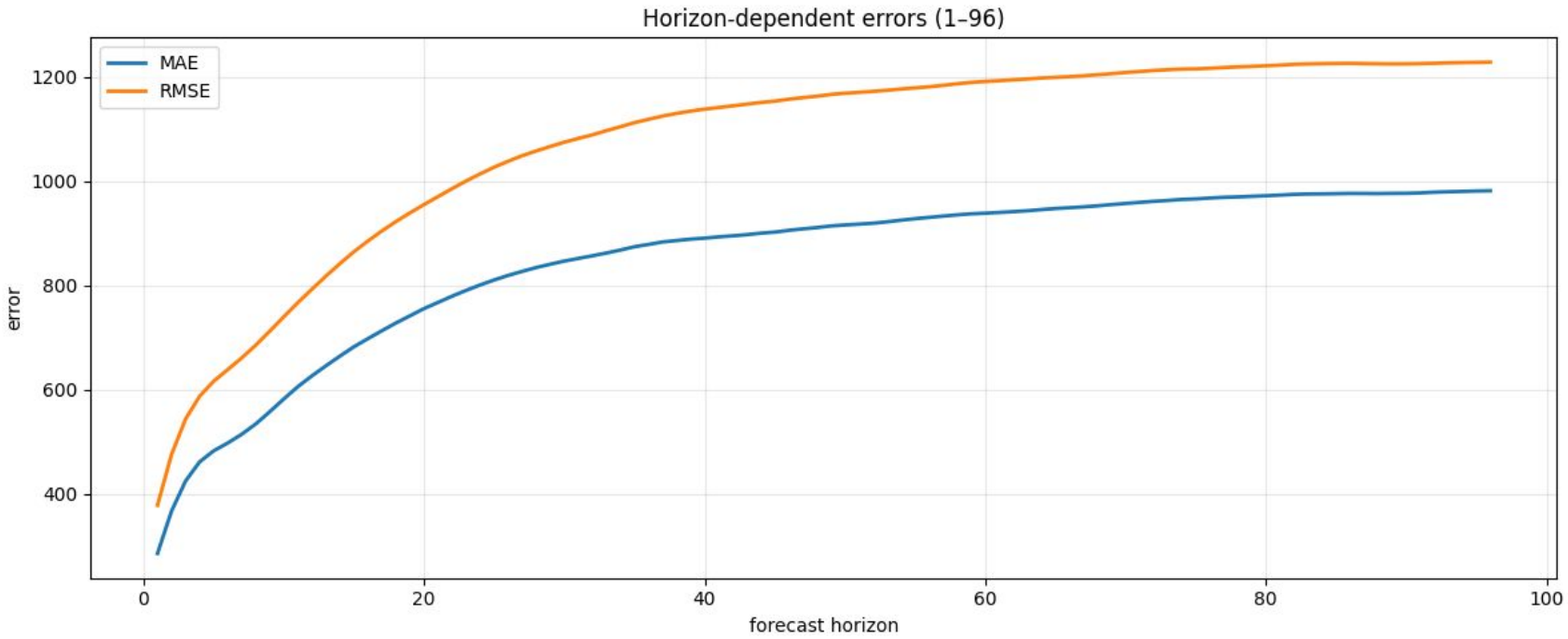
- features
  - not known in the future -> electricity load including lags and rolling mean/std
  - known in the future -> everything based in the timestamp and weather data
- train
  - use all features -> **known in the future + not known in the future**
  - using an actual RTX Series 5000 GPU with 16GB of VRAM
    - training time for the model -> 5-6 hours each time!
- validation and forecasting
  - use only features that are **not known in the future (avoid data leakage)**
  - the prediction will give you multiple horizons
  - only for the mean (0.5 quantile) or for multiple quantiles

The TFT model tolerates multicollinearity well!

Real 24-hours ahead forecast - showing quantiles



MAE & RMSE evaluation



TFT - the prediction

Window 0: |[redacted]| 96 steps
Window 1: |[redacted]| 96 steps
Window 2: |[redacted]| 96 steps
Window 3: |[redacted]| 96 steps
…

Each forecast covers the next 24 hours (96 steps), and the model repeats this thousands of times

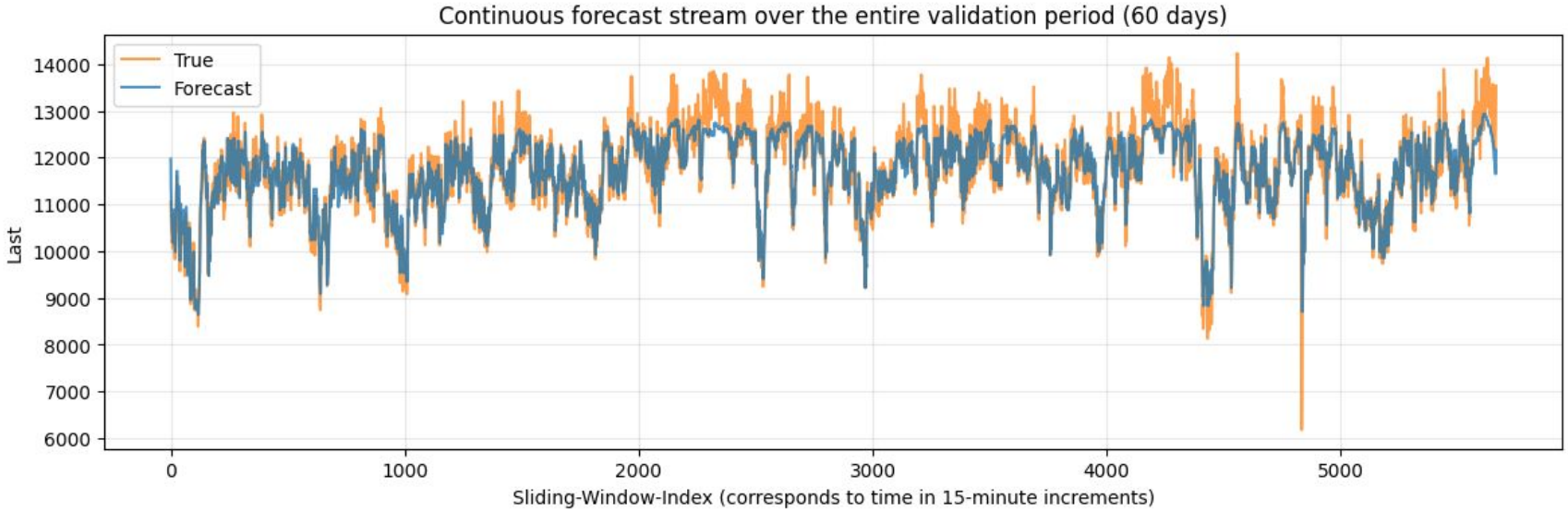
In 60 days of validation data:

→ 5665 sliding windows

→ each window predicts 96 future steps

→ total prediction shape: (5665, 96)

TFT - rolling “Next-15-minutes” prediction



predict only **15 min ahead**,

plotted for the **entire validation period** (60 days)

TFT - Interpretability and feature importance

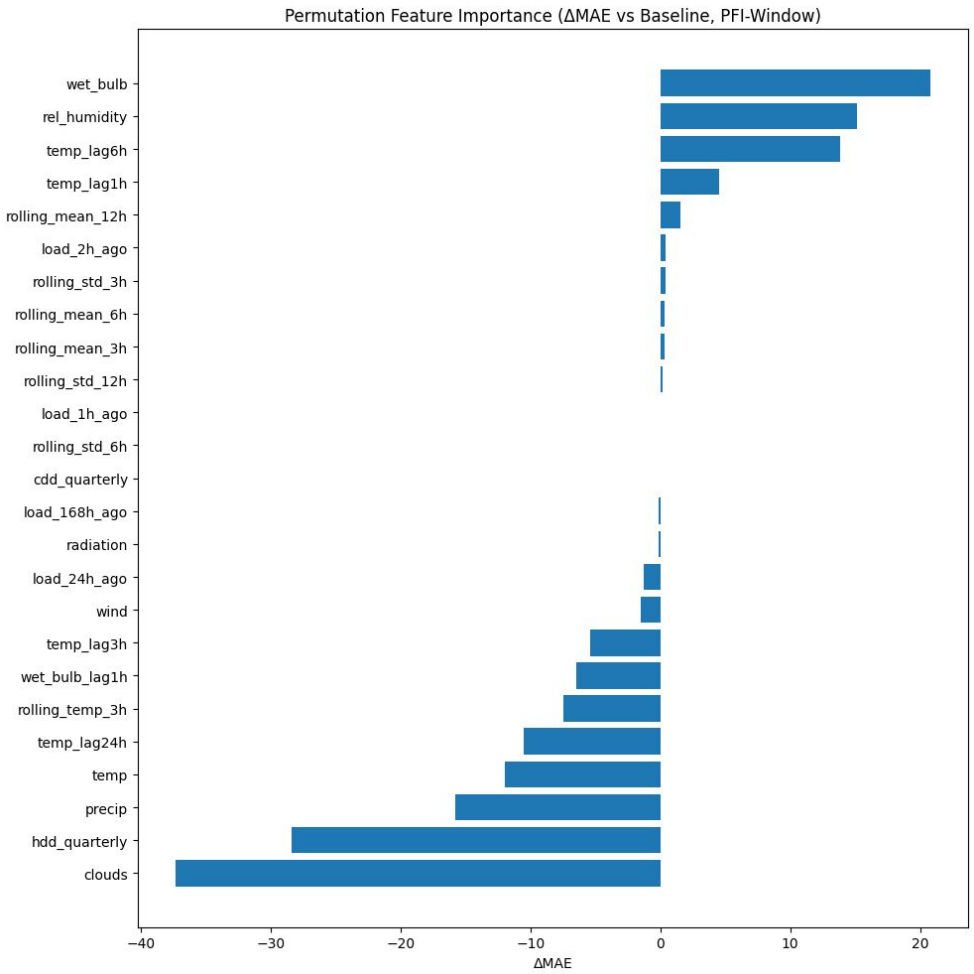
Permutation Feature Importance (PFI)

- shuffle the values of one feature (destroy it) and measure how much the forecast accuracy increases or decreases
- identify which features the TFT actively uses to forecast peak loads and daily profiles
- finding best features, not the bad ones

PFI plot

- **positive** ΔMAE
  - important feature
  - required to make good predictions

- **negative** ΔMAE
  - not a bad feature
  - random improvement of the prediction



Ready for production!

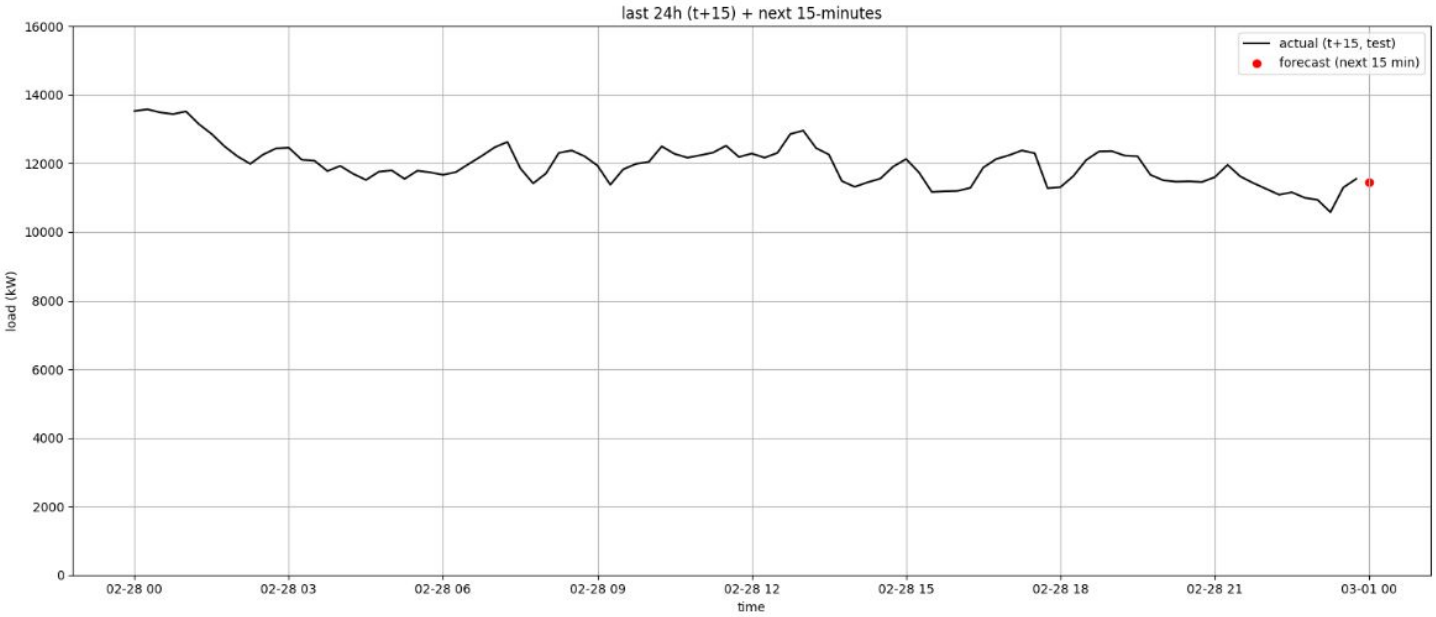
I focused on the XGBoost model

- created a function that needs:
  - path of the electricity load data to be imported (has to be at least 2 days)
  - latitude/longitude of the company
  - timezone -> UTC preferred
- running the function will automatically:
  - import the electricity load data
  - add weather data -> based on the timestamp of the imported electricity load data
  - perform data cleaning
  - call the xgb model and make a prediction for the next 15 minutes

Usage in production

input -> path of load data (at least 48h), latitude and longitude (weather)

output:



lessons learned

1. it takes a lot of time to find library/python versions that work together, while using a GPU for training
   - using virtual environments helped a lot with this
2. it is hard to learn a complex model, but it is possible
3. working with real world data is more difficult

Q&A